SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:		**Reg. No.:**	
Date:		**Duration:**	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

1. **Problem Analysis**
   - Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.
2. **Project Structure Design**
   - Design and explain the folder structure of your project repository.
   - Justify the purpose of each major folder and file included in the repository.
3. **Git Repository Initialization**
   - Create a Git repository for the project.
   - Explain the steps involved in repository initialization and describe the purpose of the essential Git files.
4. **Git Branching Strategy**
   - Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
   - Justify why the selected branching model is appropriate for the assigned project.
5. **Version Control Workflow**
   - Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
   - Explain the purpose of each Git operation performed.

6. **Commit History Analysis**
    - Present the commit history of the project.
    - Explain how commit messages follow good version control practices and contribute to project maintenance.
7. **Docker Environment Design**
    - Explain why Docker is suitable for the assigned project.
    - Identify the application components that require containerization.
8. **Dockerfile Development**
    - Design and explain the Dockerfile required to containerize the application.
    - Describe the purpose of each instruction used in the Dockerfile.
9. **Docker Compose Design**
    - Develop a Docker Compose configuration for the project.
    - Explain the services, networking, volumes, environment variables, and dependencies used.
10. **Container Deployment and Testing**
    - Demonstrate the execution of the application using Docker containers.
    - Explain the testing procedure and provide evidence that the application runs successfully.
11. **Benefits of Git and Docker**
    - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
12. **Challenges and Improvements**
    - Identify the challenges encountered during implementation.
    - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository &	Repository initialized	Repository and	Basic Git setup	Incorrect or incomplete Git

Branching Strategy(20%)	correctly with appropriate branching strategy, clear workflow, and proper repository organization.	branching strategy are mostly correct.	with limited branching implementation.	repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

CO3_AT1

AI-Based Smart Renewable Energy Asset Performance Monitoring System

NAME:RAKKESH KARAN R
REG NO:192524253
DEPT:BACHELOR OF TECHNOLOGY IN AIDS
COURSE:CSA1017 SOFTWARE ENGINEERING

1. Problem Analysis

The **AI-Based Smart Renewable Energy Asset Performance Monitoring System** is designed to continuously monitor renewable energy assets such as solar panels, wind turbines, and battery energy storage systems. Traditional maintenance depends heavily on periodic manual inspections, which may fail to identify early signs of equipment degradation or failure.

The proposed system uses **IoT sensors, cloud computing, Artificial Intelligence, Machine Learning, weather data, and analytics** to collect and analyze real-time operational information. AI models identify abnormal behavior, predict possible equipment failures, and recommend preventive maintenance actions.

Project Objectives

- Continuously monitor renewable energy assets using IoT sensors.
- Detect abnormal asset behavior using AI and Machine Learning.
- Predict potential equipment failures before they occur.
- Optimize preventive maintenance schedules.
- Analyze the effect of weather and environmental conditions.
- Identify energy-generation losses and operational inefficiencies.
- Provide real-time dashboards and alerts.
- Generate automated performance and maintenance reports.
- Reduce equipment downtime and operational costs.
- Improve renewable energy generation and asset utilization.

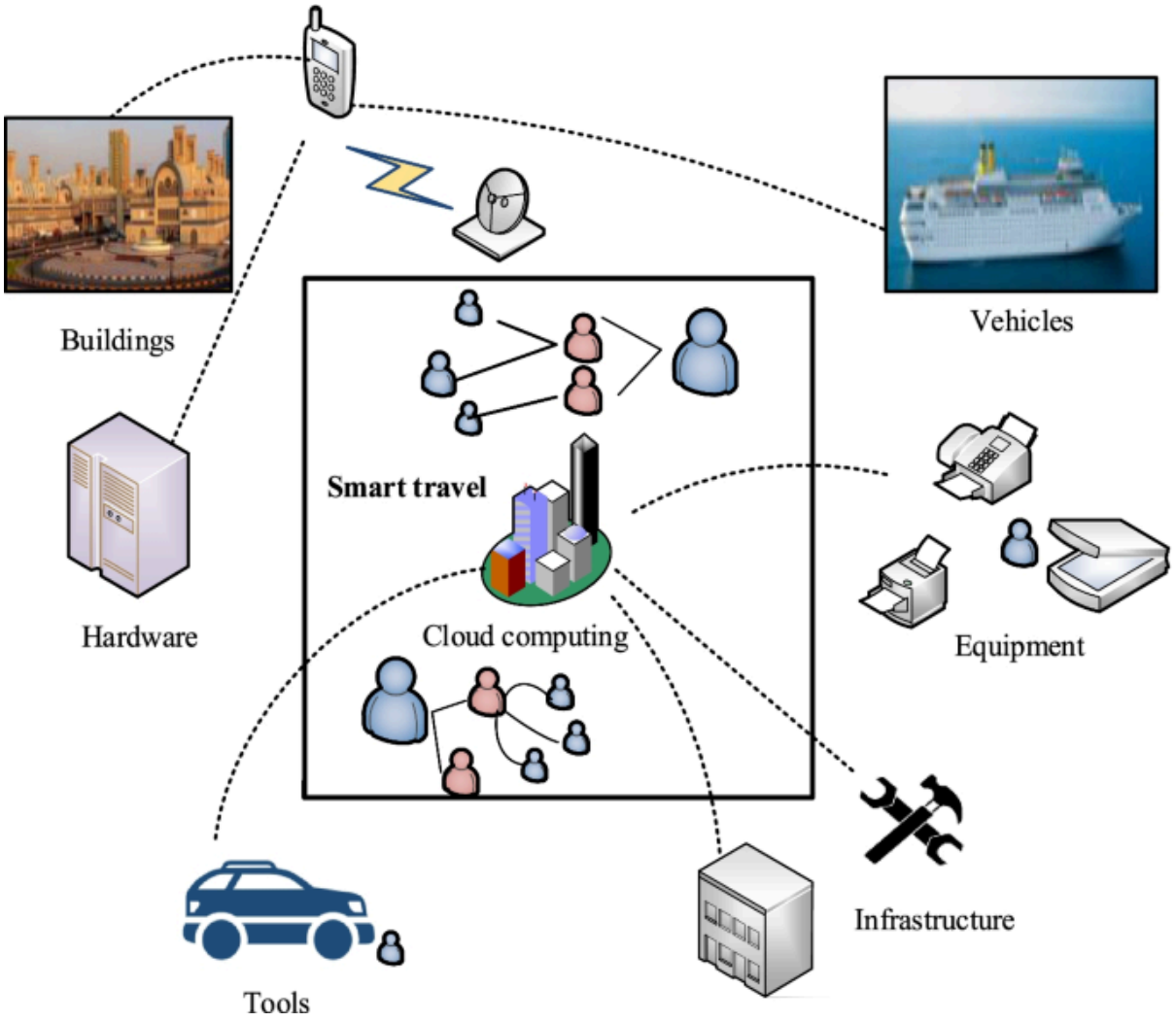
Functional Requirements

1. **User Authentication** – Operators can securely access the monitoring platform.
2. **Asset Management** – Users can add and manage solar, wind, and battery assets.
3. **Real-Time Monitoring** – The system collects sensor data continuously.
4. **Performance Analysis** – Energy generation and equipment efficiency are analyzed.
5. **AI Prediction** – Machine-learning models predict possible failures.
6. **Anomaly Detection** – Abnormal sensor readings are identified.
7. **Weather Integration** – Weather information is analyzed alongside asset data.
8. **Alert Generation** – Critical conditions generate notifications.
9. **Maintenance Recommendations** – The system recommends maintenance based on asset health.

10. **Dashboard** – Users can view asset status and performance through graphical dashboards.
11. **Reporting** – The system generates historical and performance reports.
12. **Data Storage** – Sensor and prediction data are stored for future analysis.

Expected Outcomes

The system is expected to improve renewable energy generation, reduce maintenance expenses, detect failures at an early stage, increase asset reliability, reduce downtime, and support data-driven operational decisions.



2. Project Structure Design

A suitable repository structure is:

```
renewable-energy-monitoring/
│
├── backend/
│   ├── app/
│   │   ├── api/
│   │   ├── models/
│   │   ├── services/
│   │   ├── database/
│   │   ├── ml/
│   │   └── main.py
│   ├── tests/
│   ├── requirements.txt
│   └── Dockerfile
│
├── frontend/
│   ├── src/
│   ├── public/
│   ├── package.json
│   ├── package-lock.json
│   └── Dockerfile
│
├── ml-models/
│   ├── training/
│   ├── prediction/
│   └── saved_models/
│
├── data/
│   ├── raw/
│   └── processed/
│
├── database/
│   └── init.sql
│
├── docker/
│   └── nginx.conf
│
```

```
├── docs/
│   ├── architecture.md
│   └── api-documentation.md
│
├── .gitignore
├── .env.example
├── docker-compose.yml
├── README.md
└── LICENSE
```

Purpose of Major Folders

Folder/File	Purpose
backend/	Contains server-side application logic and APIs
backend/app/api/	Defines REST API endpoints
backend/app/models/	Contains database and application models
backend/app/services/	Implements business logic
backend/app/ml/	Connects AI/ML functionality with the backend
backend/tests/	Contains backend test cases
frontend/	Contains the monitoring dashboard
ml-models/	Stores training and prediction-related ML code
data/	Contains raw and processed datasets
database/	Contains database initialization scripts
docker/	Contains supporting Docker configuration
docs/	Contains project documentation
.gitignore	Prevents unnecessary or sensitive files from being committed
.env.example	Provides an example of required environment variables
docker-compose.yml	Defines and connects application containers
README.md	Provides project setup and usage instructions

LICENSE	Defines project usage and distribution terms

This structure separates frontend, backend, machine learning, database, and deployment components, making the project easier to maintain and develop collaboratively.

---

3. Git Repository Initialization

Git provides distributed version control for tracking changes in the project.

Repository Initialization Steps

mkdir renewable-energy-monitoring
cd renewable-energy-monitoring

git init

git add .
git commit -m "Initial project structure"

git branch -M main

git remote add origin <repository-url>

git push -u origin main

Important Git Files

.git/
Created automatically by git init. It stores Git's internal repository information, including commits, branches, and configuration.

.gitignore
Specifies files and directories that should not be tracked.

Example:

.env
__pycache__/

node_modules/
*.log
*.pyc
models/*.pkl

.git/config
Contains repository-specific Git configuration.

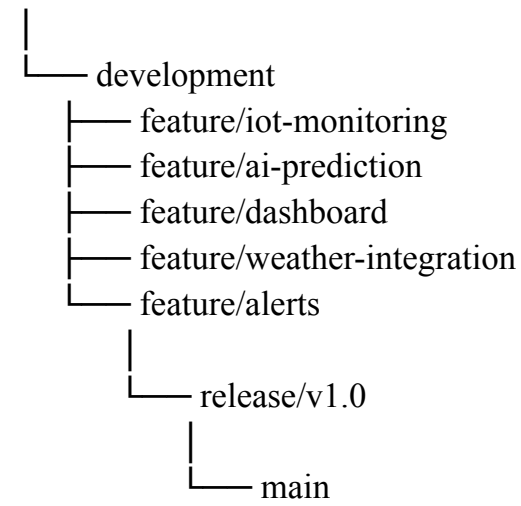
.git/HEAD
Identifies the currently checked-out branch.

The Git repository allows every change to the renewable energy monitoring system to be tracked and restored when necessary.

---

4. Git Branching Strategy

A **Git Flow-style branching strategy** is suitable for this project.



Main Branch

Contains stable and production-ready code.

Development Branch

Contains integrated features that are being prepared for the next release.

Feature Branches

Used to develop individual features independently.

Examples:

feature/iot-monitoring
feature/ai-prediction
feature/dashboard
feature/weather-api

Release Branch

Used for final testing, bug fixing, and preparation of a release.

Example:

release/v1.0

Hotfix Branch

Used to quickly fix critical problems in the production version.

Example:

hotfix/critical-alert-bug

Justification

This strategy is appropriate because the project contains several independent components such as IoT monitoring, AI prediction, dashboards, alerts, and weather integration. Developers can work independently without directly modifying stable production code.

---

5. Version Control Workflow

A typical workflow is:

Create Issue
↓
Create Feature Branch
↓
Develop Feature
↓
Test
↓
git add
↓
git commit
↓
Push Branch
↓
Code Review
↓
Merge into Development
↓
Release Testing
↓
Merge into Main

Example Git Operations

Create a feature branch:

```
git checkout development
git pull origin development

git checkout -b feature/ai-prediction
```

Make changes and commit:

```
git add backend/app/ml/
git commit -m "Add equipment failure prediction model"
```

Push the branch:

git push -u origin feature/ai-prediction

Merge into development:

git checkout development
git pull origin development
git merge feature/ai-prediction
git push origin development

Purpose of Git Operations

Operation	Purpose
git init	Creates a Git repository
git status	Shows modified and untracked files
git add	Stages changes
git commit	Creates a permanent version snapshot
git branch	Creates or manages branches
git checkout	Switches branches
git merge	Combines changes
git pull	Downloads and integrates remote changes
git push	Uploads local commits
git log	Displays commit history

Conflict Resolution

If two developers modify the same section of a file, Git may report a conflict.

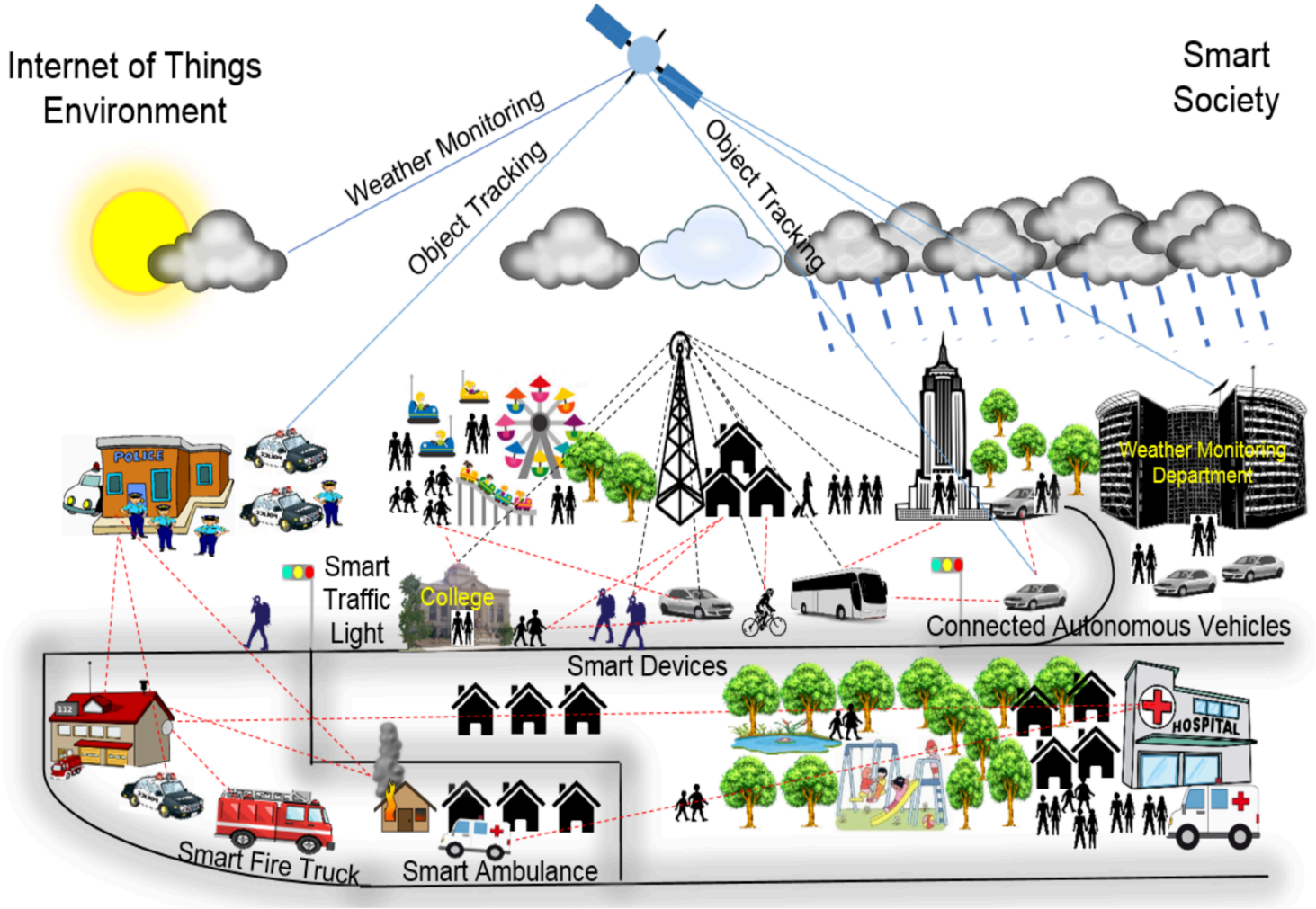
git pull origin development

After manually resolving the conflicting code:

git add .
git commit -m "Resolve dashboard integration conflict"

git push origin development

This ensures that conflicting changes are reviewed before they become part of the shared branch.



6. Commit History Analysis

A meaningful commit history could be:

a82f91d Add Docker Compose configuration
b51e720 Implement renewable asset dashboard
c34a610 Add AI equipment failure prediction
d72f431 Add IoT sensor data API
e18c902 Create database schema
f63b205 Add backend project structure
g91a432 Initialize project repository

Good Commit Message Practices

Commit messages should:

- Be short and descriptive.
- Explain what was changed.
- Use an action-oriented format.
- Avoid vague messages such as update, changes, or final.
- Represent one logical change where possible.

Examples:

Add IoT sensor monitoring API
Implement asset health prediction
Add weather data integration
Fix incorrect battery status calculation
Add renewable energy dashboard
Update Docker deployment configuration

A well-organized commit history makes it easier to identify when a feature was introduced, understand project evolution, investigate bugs, and revert problematic changes.

---

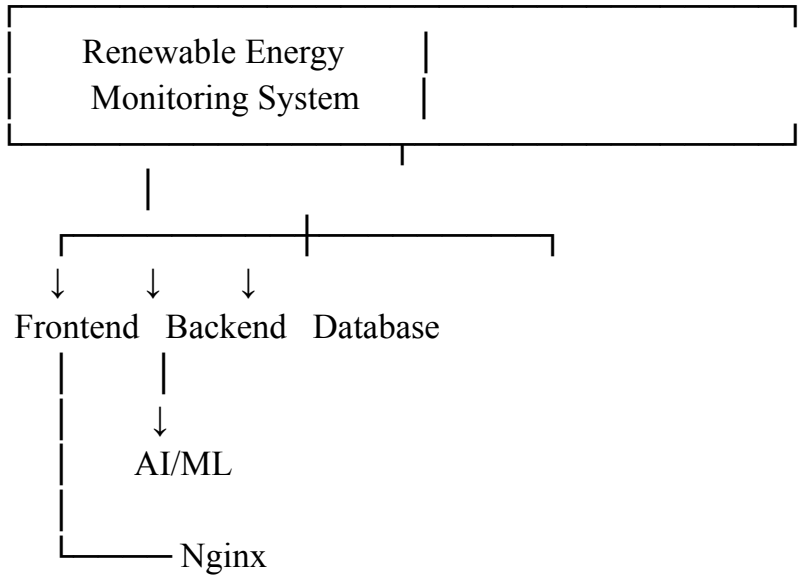
7. Docker Environment Design

Docker is suitable because the project contains multiple technologies and services, including the frontend, backend, database, and AI components.

Without containers, developers may need to manually install compatible versions of Python, Node.js, databases, libraries, and ML dependencies.

Docker provides a consistent environment for development, testing, and deployment.

Components Suitable for Containerization



The major containers can be:

1. **Frontend Container** – Hosts the monitoring dashboard.
2. **Backend Container** – Provides APIs and application logic.
3. **Database Container** – Stores asset and sensor information.
4. **ML Service** – Performs prediction and anomaly detection.
5. **Nginx Container** – Acts as a reverse proxy if required.

---

8. Dockerfile Development

A sample backend Dockerfile is:

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY app/ ./app/

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Dockerfile Instructions

FROM
Selects the Python base image.

WORKDIR
Sets /app as the working directory inside the container.

COPY
Copies project files into the container.

RUN
Installs Python dependencies.

EXPOSE
Documents the port used by the backend.

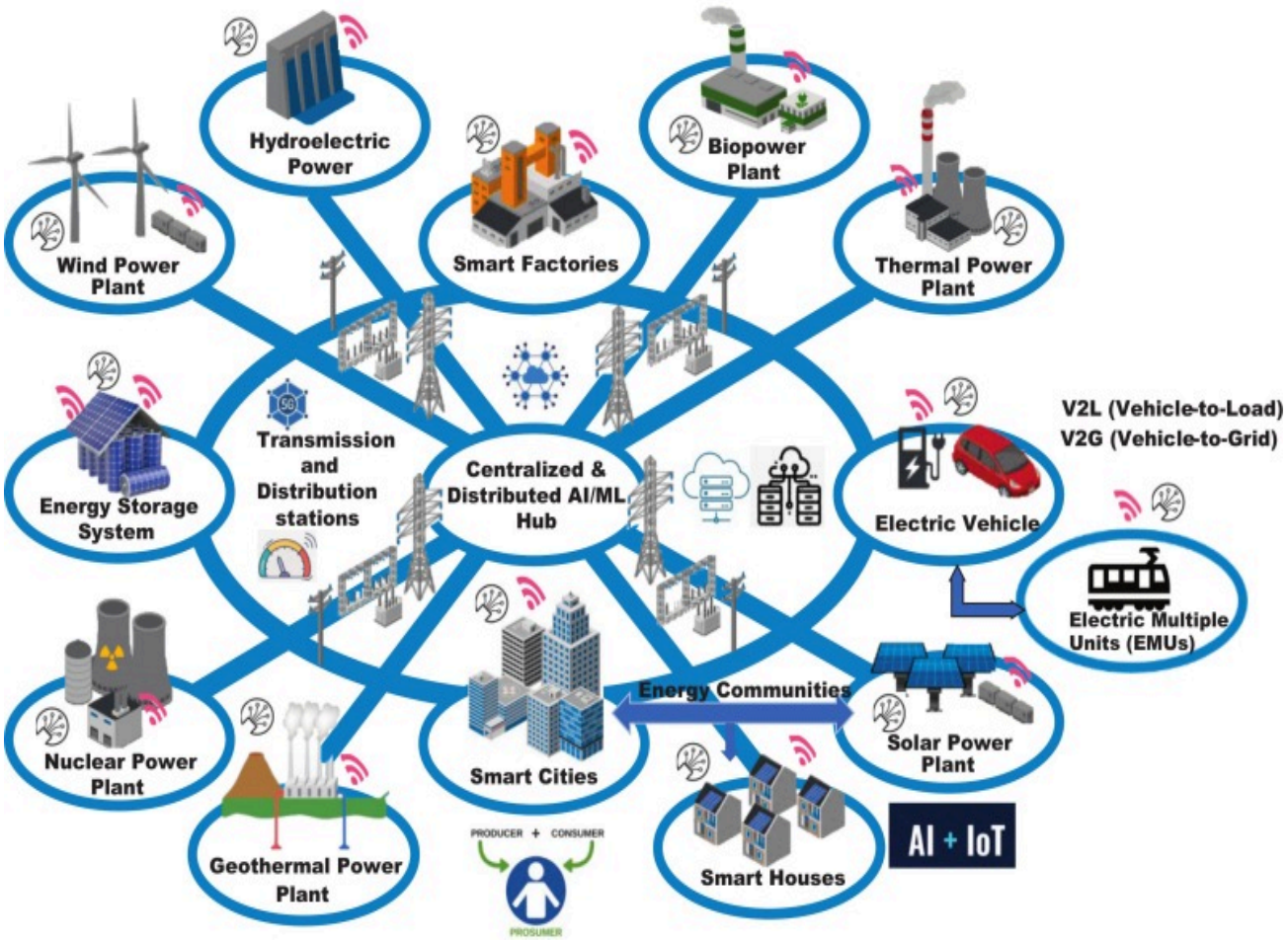
CMD
Starts the application when the container runs.

Frontend Dockerfile

```
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

RUN npm run build

EXPOSE 3000
```

CMD ["npm", "start"]

This Dockerfile creates a reproducible environment for the renewable energy monitoring dashboard.



9. Docker Compose Design

A sample docker-compose.yml is:

```
services:


  backend:
    build: ./backend
    ports:
      - "8000:8000"
    environment:
                                                                DATABASE_URL:
postgresql://energy_user:energy_pass@database:5432/energy_db
    depends_on:
      - database
    networks:
      - energy-network

  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
    depends_on:
      - backend
    networks:
      - energy-network

  database:
    image: postgres:16
    environment:
      POSTGRES_DB: energy_db
      POSTGRES_USER: energy_user
      POSTGRES_PASSWORD: energy_pass
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - energy-network

volumes:
```

postgres_data:

networks:
energy-network:

Services

- **Backend:** Runs the application API.
- **Frontend:** Runs the monitoring dashboard.
- **Database:** Stores system data.

Networking

The energy-network allows containers to communicate with each other using service names.

For example:

backend → database:5432
frontend → backend:8000

Volumes

postgres_data:

The volume ensures that database data remains available even if the database container is removed and recreated.

Environment Variables

Database configuration is passed through environment variables rather than hard-coding configuration inside application code.

For production deployment, passwords should be stored securely rather than directly in the Compose file.

Dependencies

depends_on:

ensures that the required database service is started before the backend service.

---

10. Container Deployment and Testing

The complete system can be started using:

docker compose build
docker compose up -d

Check running containers:

docker compose ps

View logs:

docker compose logs

Test the backend:

curl http://localhost:8000

The dashboard can be accessed through:

http://localhost:3000

Testing Procedure

1. Build all Docker images.
2. Start the containers.
3. Verify that all containers are running.
4. Check backend logs for errors.
5. Verify database connectivity.
6. Access the frontend dashboard.
7. Send sample sensor data to the backend.
8. Verify that sensor information is stored.
9. Test AI prediction functionality.
10. Verify that alerts are generated for abnormal readings.
11. Confirm that dashboard values are updated.
12. Stop and restart containers to verify persistence.

Expected Evidence

Successful deployment should show:

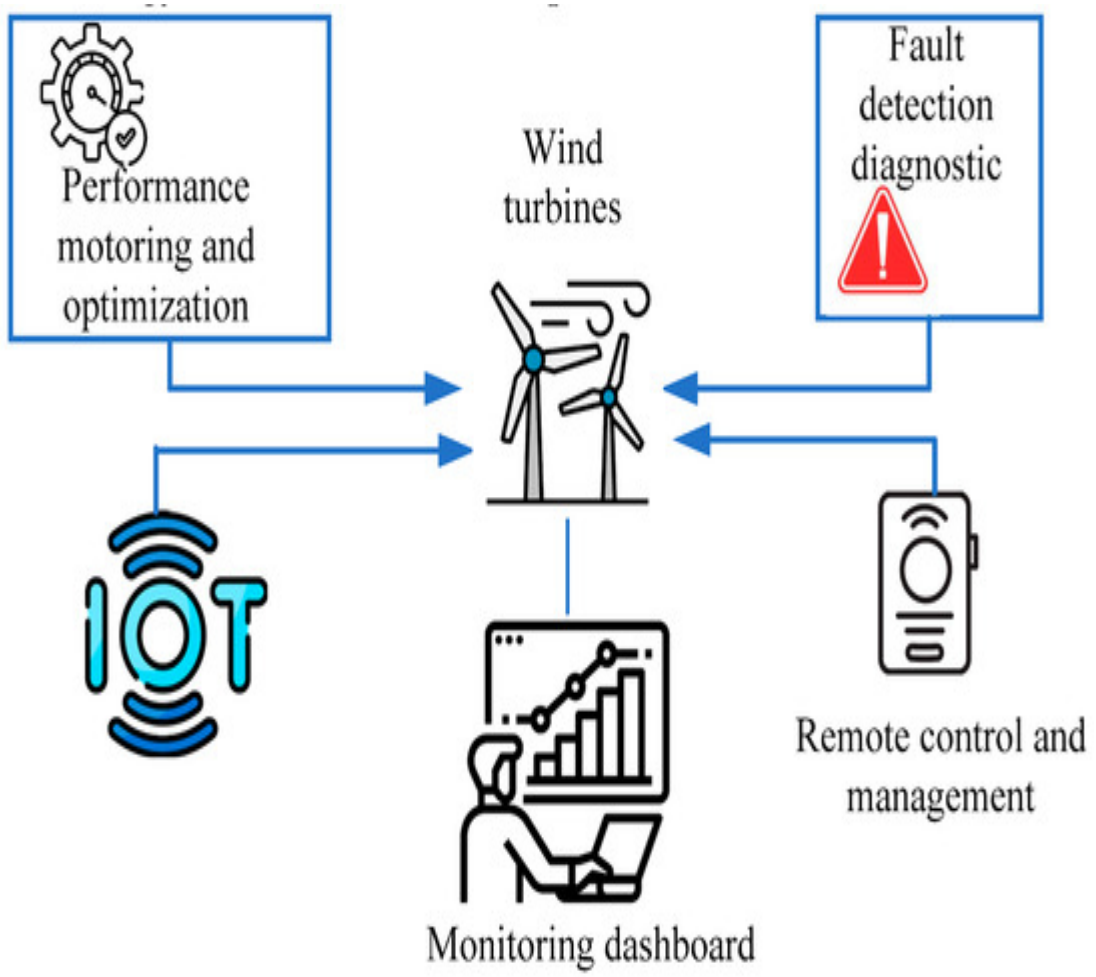
CONTAINER STATUS

frontend Running
backend Running
database Running

The dashboard should display renewable asset information such as:

Solar Plant : Online
Wind Turbine : Healthy
Battery System : Normal
Energy Output : 85%
Asset Health : Good
Alerts : 1

This demonstrates that the application is successfully running inside Docker containers.



11. Benefits of Git and Docker

Area	Git Benefit	Docker Benefit
Collaboration	Multiple developers can work simultaneously	Same environment for all developers
Version Management	Tracks every source-code change	Versions application environments
Portability	Repository can be cloned anywhere	Containers run consistently across systems
Deployment	Maintains deployment-related files	Simplifies application deployment
Scalability	Supports organized development branches	Allows services to be replicated
Maintainability	Provides history and rollback	Isolates application dependencies
Testing	Changes can be tracked and reviewed	Provides reproducible test environments
Reliability	Previous versions can be restored	Reduces environment-related failures
CI/CD	Integrates with automated pipelines	Supports automated image building and deployment

Combined Advantage

Git manages the **source code and its history**, while Docker manages the **application environment and deployment**.

Together they provide a reliable development workflow:

Git
↓
Source Code Versioning
↓
Testing
↓
Docker Image

↓
Container
↓
Deployment

---

12. Challenges and Improvements

Challenges

1. Managing Large Sensor Data
Renewable energy systems generate large volumes of real-time sensor data.

2. ML Model Integration
Integrating trained machine-learning models with the backend can introduce dependency and performance issues.

3. Container Dependency Management
Different services may require different software versions.

4. Database Persistence
Sensor data must remain available when containers are restarted.

5. Network Communication
Frontend, backend, database, and ML services must communicate correctly.

6. Git Merge Conflicts
Multiple developers working on the same files may create conflicts.

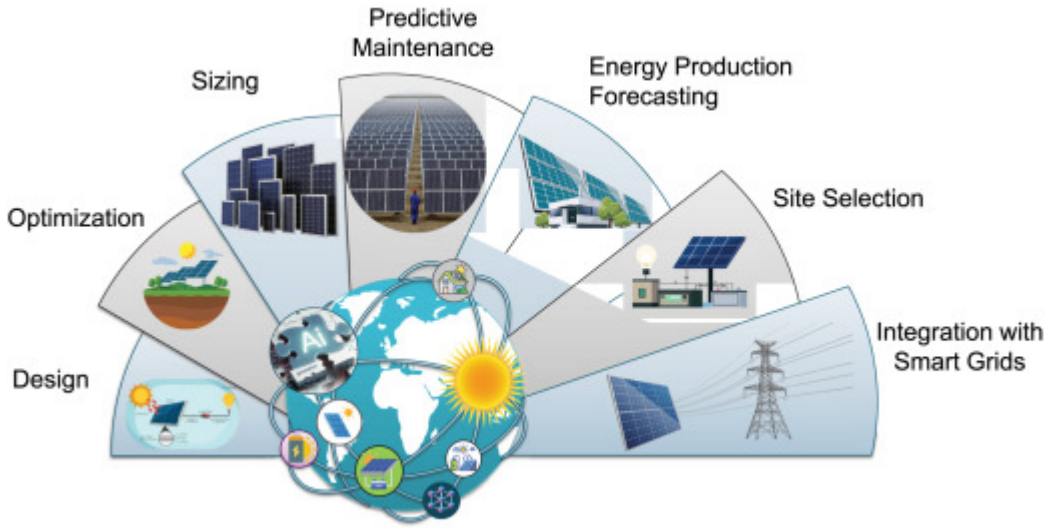
7. Sensitive Configuration
Database passwords, API keys, and cloud credentials should not be committed to Git.

8. Resource Requirements
AI/ML processing may require significant CPU, memory, or GPU resources.

Improvements

- Implement automated CI/CD pipelines.
- Add automated unit and integration testing.
- Use Docker health checks.
- Use Docker image optimization techniques.
- Add centralized logging and monitoring.

- Use secure secret management.
- Implement container vulnerability scanning.
- Introduce automated database backups.
- Add Kubernetes for large-scale deployments.
- Improve AI models using continuously collected operational data.
- Add real-time notification services.
- Implement automated model retraining.
- Use cloud-based storage for large historical datasets.
- Add role-based access control for different system users.



Conclusion

The **AI-Based Smart Renewable Energy Asset Performance Monitoring Syste**can benefit significantly from Git and Docker. Git provides structured version control, collaborative development, branching, rollback, and project history management. Docker provides isolated and reproducible environments for the frontend, backend, database, and AI services.

The combination of **Git + Docker + IoT + AI/ML + cloud technologies** provides a scalable foundation for developing and deploying a reliable renewable energy monitoring platform. The resulting system can support predictive maintenance, reduce operational downtime, improve energy generation, and help renewable energy operators make faster and more informed decisions.